

•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- Feedzai Random Forest

On this page

Feedzai Random Forest

Was this page helpful? 

Feedzai Random Forest (FZRF) is a machine learning algorithm developed by Feedzai. It is an enhanced variation of the [original Random Forest algorithm](#).

Random forests use labeled data to learn its patterns. A random forest consists of a typically large number of decision trees, where each decision tree is a model. Each decision tree model is created by feeding each tree a random subset of the data (i.e. bagging).

In this topic you can learn about the parameters and the usage of Feedzai Random Forest.

Parameters

You can learn more about each parameter of FZRF in the following table:

Parameters	Description	Default values	Possible values	Notes
In-bag factor	This parameter tells you which percentage of instances are used to train a particular tree, while the rest is used to calculate the out-of-bag error.	0.65	Double between 0 and 1.	See FAQ's question number 2 to learn more about how the bagging is performed.
Maximum depth of trees	The number of nodes in the longest path from the root of the tree to a leaf. The higher the number, the larger the model will be. Large models mean more splits, meaning they capture more information about the data. However, as mentioned, they result in large sized models that can reach GBs. Also, a high number of maximum depth of trees is likely to lead to overfitting.	20	Positive integer.	This can be a number from 1 to the number of instances in the training set. In practice, splits don't separate just one transaction from the others, so depending on the rate of the target class and features being used to train, you won't see a considerable difference in model performance (i.e., if you change the maximum depth from 100 to 200). However, the size of the model may change considerably so a small number is advisable.
Number of trees	The number of decision trees in the forest.	100	Positive integer.	There are diminishing returns, and training time increases linearly with the number of trees. Typically this parameter is set between 500 or 1000, but only if the dataset isn't particularly big.
Feature selection function	The metric that defines how to pick the feature for the next split.	Information gain	Information gain: The next split will be the one that maximizes the information gain, i.e., that splits the two classes better. This means these feature selection functions give you a concrete number on how well you separated your two classes (e.g. fraud vs non-fraud) after a split. Your goal is to be able to separate the two classes as best as possible.	
Minimum instances for split	For a split, the selected node has to have at least this number of instances in its subtree.	2	Positive integer.	The typical value is 10 or 20. If you select a number equal or bigger than the number of transactions, then you never do any split.
Number of candidate features	The number of features to be used in each node as candidates for the next split.	0	Positive integer, between 0 and the total number of features in the dataset.	The default value (i.e., 0, zero), means that the square-root of the total number of features will be used.
Prior function	This parameter is used only when estimating the distribution of a node that has no transactions.	Empty	Empty: Assigns an equal probability for each class. If it's a	The estimation happens when scoring an event that has null values in a feature that is used to split nodes in the trees.

Parameters	Description	Default values	Possible values	Notes
			fraud vs non-fraud scenario, 50% for each. Random: assigns a random probability for each class. Parent: uses the probability distribution of the parent node.	
Split method	The way the algorithm splits categorical nodes into new branches.	One vs Others	One vs others: the split is done into two new branches: one category branch vs an "others" branch. This one results in longer training time. All vs all: the split is done into as many branches as the number of categories. Typically, you will not obtain better results from this value, as it's prone to overfit if there are a lot of categories and it also requires larger disk space, which potentially means trouble.	

FAQ

What is the condition for the nodes to be split?

What is the condition for the nodes to be split, is it '<' goes left and '>=' goes right or '<=' goes left?

It shouldn't make any difference if you are trying every split.

Is bagging with or without repetition?

Without repetition for each tree. With repetition "across trees", as shown in the figure below.

This means that a tree will never look at the same instance twice, but multiple trees in the same "forest" will most likely see overlapping instances.

How do null values impact the computation of information gain, especially with categoricals?

If you are trying to split on a categorical and an instance has a null in that field, you must count the instance for the total. Yet you shouldn't count it for the specific subset when calculating the information gain.

A few more notes regarding FZRF and null values:

- FZRF will ignore any instance where the class label is missing. This means that the whole instance is dropped during training.
- FZRF will use instances where the features have missing values up to the first split using that feature (i.e. for each tree/branch).

The consequence of the last point is that when a feature with very high number of missing values is being tested for splitting, it may **have very high information gain**. This will lead the model to select the feature, hence dropping most of the instances in the training data.

Is the data replicated into memory for each tree due to bagging?

Yes, because trees are trained in parallel and each in-bag dataset is a partition of the initial data.